淘特端侧模型从微调到应用的全链路实践

原创 营销&交易技术 大淘宝技术 2026年10月15日 20:50 浙江 7人



本文总结了淘特 App 从 3 月到 8 月在端侧模型建设上的全链路实践。为解决用户实时意图识别及云端大模型延迟与成本问题，团队选用 QWen3-0.6B 作为基础模型，通过两代数据集迭代将真机准确率提升至 80% 以上。在工程实现上，采用 LoRA 微调、基于 ShortGPT 的深度剪枝以及 AWQ 量化技术，大幅优化了模型体积与推理性能；同时结合 Prompt 精简、Prefix Cache 和增量推理等手段显著降低了端到端延迟。最终该端模型成功落地于信息流重排序与流失前权益挽留场景，实现了 GMV 的正向增长，并针对 CDN 下载等隐形成本提出了按需下载等优化策略。

01 写在最前面

记录和整理了从今年 3 月到 8 月在淘特 App 上做的端模型相关探索，核心内容包括

- 训练平台和模型的选型 – 端侧模型的特殊性对平台选择的考量；
- 微调 – 从无到有以及从有到优的一些经验；
- 剪枝 – 牺牲极少准确率获得模型大小优化和推理性能的大幅增加；
- 量化 – 量化方式和参数对模型大小和真机准确率的影响；
- 推理性能探索和优化 – Prefix 等常见优化方向以及特定场景下的增量推理优化介绍；
- 场景落地 – 淘特落地业务及其 AB 数据；
- 小模型接入的隐形成本 – CDN 等隐形成本考量；

为后续有兴趣在端侧接入小型模型的同学提供一些参考。

02 背景：为什么探索端侧模型

原因一 – 用户实时意图的探索从未停止：

翻开 ATA 搜索"用户实时意图"，会发现从 2013 开始到现在这个方向一直没有停止过，从纯工程实现到神经网络再到 LLM 大模型，我们一直在持续探索更优更准的用户实时意图获取方式。

原因二 – 端模型探索时机成熟：

AI 的浪潮推进到端侧。22 年 11 月 ChatGPT 证明其商业价值后 AI 浪潮以惊人的速度向前推进，到 26 年 4 月，集团内几乎所有技术团队都在拥抱 AI，集团内信息流 & 直播等团队基于 QWen 均已开始端模型的探索和应用。4 月 Gemma E2B 的推出带给端模型更多可能。

端模型基建成熟。AI 浪潮的推进让集团内公共训练环境、基础模型微调、量化等基建日渐成熟，加上 Claude Code / Cursor 等工具可以快速理解/改造 微调用的 LLaMaFactory、量化用的 AliNNPrivate等工程，原先站在大模型领域外围的同学已可以快速切入。

原因三 – 业务诉求：

怎么识别这个用户在 App 有较强的购物诉求还是随意浏览？用户有意图但快离开 App 时我们是否可以更精准感知并用权益挽留？目前有很多云端基于 LLM 的方案，速度和 Token 成本结合端侧模型是否有办法有更优的解法？

基于以上核心因素，淘特在今年 3 月底开始端模型的探索、应用和落地。

03 做什么：探测什么意图

结合信息流和权益方法的场景，反复权衡和实验后，最终针对进端购物的用户确定让端模型识别 3 种用户意图：

- 闲逛：用户无目的随意浏览。
- 比价犹豫：用户围绕某一单品反复浏览对比。
- 场景购物：围绕某一场景选购，例如露营(帐篷、天幕、GTI 桌)、钓鱼(鱼钩、鱼线、鱼竿、鱼饵)。

场景应用：

- 场景1 – 用户有购物意图但未下单准备离开时发券，用权益钩子提升用户下单意愿。
- 场景2 – 针对用户意图将未曝光 Feeds 基于意图重排序，感兴趣卡片靠前展示，提升点击率。

以某条动线为例：

```
"【行为序列】商详页(万全通用空调遥控器能适用格力美的海尔海信科龙小米华凌志高
TCL,cat:50011972.50011866.50016107,66s) → 首页(16s) → 商详页(空调遥控器通
用格力美的松下海信海尔奥克斯TC春兰日立志高长虹东芝夏普格兰仕伊莱克斯惠而浦大金
三菱,cat:50008090.50011826.350116,5s) → 首页(4s) → 商详页(家用空调遥控器万
```

```
通用款全部能适用于格力美的华凌海尔TCL奥克斯，
¥11.64,cat:50008090.50011826.350116,68s）→ 首页(1s）→ 商详页(【超1000次加
购】适用美的空调遥控器通用款全部型号kfr23gw 35rn02d a华凌小天鹅省电星冷俊星挂
机柜机中央制器摇控板,¥5.46,cat:50008090.50011826.350116,48s）→ 商详页(夙就
适用于美的KF-26GW/Y-DH400(D3) 省电星大1匹单冷空调遥控器遥控板，
¥11.64,cat:50011972.50011866.50016107,12s）→ 商详页(适用美的空调遥控器万能
通用省电星智弧冷静星冷俊星冷慧星高能星，
¥8.23,cat:50008090.50011826.350116,2s）→ 商详页(【官方正品】适用美的空调遥
控器通用KT-A899K款全部型号中央省电星小白鲨小天鹅柜机智弧摇控板华凌2906，
¥9.75,cat:50008090.50011826.350116,43s）→ 商详页(适用于美的空调遥控器万能通
用华凌智弧省电星智行冷俊星小天鹅酷金kfr-35gw26gwr51d挂机中央台式专用摇控
板,cat:50008090.50011826.350116,1s)"
```

预期模型根据动线推测意图及用户浏览商品的关键词：

```
"比价犹豫|空调遥控器;美的;通用款;省电星;华凌"
```

推荐侧收到意图做 Feeds 的重新排列，空调遥控器相关匹配商品前置。

权益中心获取用户意图判断是否需要触发发券逻辑，离端前提升下单意愿。

04 怎么做：数据集建设、微调、量化、部署、推理、应用

端侧模型整体架构



整体架构主要分成 4 层，各层主要职责如下：

- 模型层：通过训练数据集的清洗/优化、微调参数调整、量化参数调整等方式优化模型准确率；
- 推理引擎层：引擎原理的熟悉 & 推理性能优化，核心指标是端到端延迟优化；
- 端侧 AI Lab：模型管理和下载、用户UT获取和组装、引擎调用和意图结果分发；
- 业务层：意图结果监听 & 使用；

下面就各层中重点内容展开。

4.1 微调数据集建设和迭代

模型输出(意图 + 关键词)确认后，最开始也最重要的一步是如何创建高质量数据集，数据集质量越高，LoRA 微调的效果越好，质量优于数量。从淘特经验看，第一个数据集我们的准确率只有43%，后续建设和迭代是一个持续调优的过程，N 版本之后建设到 91.8%。

数据集建设整体分成两个部分展开，一是基于 UT 链路埋点 从 0 到 1 建设的过程，二是灰度后利用上线数据优化的过程。

4.1.1 第一代数据集：从原始埋点合成训练数据

端模型冷启动阶段无线上推理数据可用，从埋点日志出发：规则引擎做硬过滤与预标注，云端大模型蒸馏出标签，有疑问项人工校对并标注，生成第一个版本的训练数据。

流程如下：



第一代建设中几个要点：

- **废弃 CoT**：第一代在这个版本之前还有一个带 CoT（思维链)的版本，原想法是蒸馏云端模型思考方式喂给端模型增加输出准确性。CoT 被废弃原因：

  1、幻觉：给小模型增加 CoT 后，返回出现幻觉的概率大增，整体准确率不到 60%；

  2、耗时：在给出最终结果之前 CoT 的输出非常耗时，影响链路响应时间。用户行为输入到意图生成往往 1 分钟过去了；

- **云端标注从单模型优化成双模型**：起初只有 Qwen3-Max，为增加数据有效性防止单模型偏好，引入交叉验证模型。50 条分层 benchmark 中 Kimi K2.5 与其一致率最高（72%，优于 GLM-5 70%、DeepSeek V3.2 68%）且延迟可接受，引入作为第二标注源做全量交叉标注：一致自动采纳，分歧转人工。以 0420 批次 10300 条为例，自动采纳 69.3%。

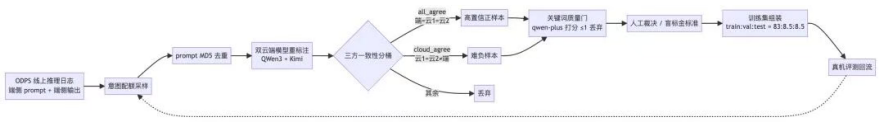
第一代方案的缺陷：

- **天花板等于教师模型规则**。标签来自云端教师模型与规则引擎硬过滤，训练本质是蒸馏教师模型判断，学不到教师模型以外的判别力；
- **数据集分布和线上不一定一致**。意图分布人为设定（场景购物 7% / 比价犹豫 15% / 闲逛 78%），边界场景的补充也不足，该数据集具体线上真实用例推理的数据、分布比例会有差异，这种差异可能导致离线测评数据和线上真是准确率数据的差异；

第一代数据集经过多轮迭代， DSW 环境准确率 84%，量化后真机准确率 76%，7月上旬开始第一批少量灰度。

- 4.1.2 第二代：从线上真机推理日志回流重建

当第一个版本的模型灰度放量后，线上用户真实的数据逐渐积累，在放量到 10W 级别用户后，收集线上真实数据，清洗并按线上意图的分布比例重建数据集。流程如下：



线上数据集清洗过程中的部分要点：

- **三方一致性** – 端侧输出与两个云端模型标注做三方比对：

  – 三者一致为高置信正样本，直接入集；

  – 两个云端一致而端侧不同单列为困难样本——这正是模型当前的错误边界，定向补入收益最高；

Tips：QWen3-Max 与 Kimi 2.5 两模型自身偏好也不一致，不能完全依赖云端双模的共识，需要抽样一定样本人工检测；

- **关键词质量保障** – 非闲逛样本由 qwen-max 按 0~3 分评估「候选关键词是否准确概括行为序列中的核心商品或场景」，≤1 丢弃。防止 LLM 的幻觉关键词进入训练数据；
- **人工标注** – 三个模型都不一致的样本由人工标注。人工标注文件不展示任何模型预测，只给行为序列，避免锚定效应。将人工标注后的部分数据集补充到训练数据集。目前人工校验耗时较大，每次只标注 50 条数据；

第二代数据集经过迭代 DSW 环境准确率 91%，量化后真机准确率 80%。

4.2 模型训练平台的选择

完成数据集建设后，下一步是挑选一个后训练平台，端侧除了微调还涉及微调后的模型合并、剪枝、下载、转换、量化，对平台的选择有更多诉求。有 2 个重点因素需要考虑：

1、是否支持微调后的转换、量化、下载。如不支持无法进行后续转换等操作，无法部署到端侧；

2、GPU 型号和性能。从目前经验看到老卡和新卡微调后结果 DSW 环境差异不大但量化后准确率差异明显。尽可能选择更高端 GPU；

尝试了 ideaLab、ModelScope 等诸多平台后，最终我们选择 ModelScope 平台 + LLaMaFactory 搭建微调环境。部分平台记录的坑：

- 不支持 QWen3-0.6B 小模型，只支持 2B 及以上参数模型；
- 不支持微调后下载，导致无法使用 MNN 转换和量化；
- 部分平台如果用公共资源排队时间超长；
- 部分平台显卡较老导致量化后效果较差；

不同 GPU 硬件量化后真机同一套 val 数据集 benchmark 准确率差异(最好和最坏准确率差异超过 21%)：

训练平台	GPU 卡	真机 CPU 推理准确率	真机 OpenCL 推理准确率
PAI	英伟达 卡1	64.4%	63.0%
ModelScope	英伟达 卡2	70.2%	73.8%
ModelScope	AMD 卡1	81.6%	84.4%

建议：资源紧张型的团队可以选择免费或低成本环境先搭建和走通整个流程，之后如果有条件切换到高阶 GPU 做微调等操作，之后对比同一数据集下测评结果。

4.3 模型的选择

目前业界在移动端部署最广泛的模型主要集中在 0.5B 到 3B 参数规模。业界专为移动端和边缘计算优化的小模型家族包括 Google 的 Gemma 系列、微软的 Phi 系列、阿里巴巴的 Qwen 系列（及其衍生）、以及由社区驱动的 MiniCPM、SmolLM、TinyLlama 等多个分支，横向对比如下：

模型名称	参数量 (约)	关键技术/特点
MiniCPM	4.1B	原生稀疏架构InfLLM 2.0，支持长文本处理；集成FR-Spec轻量投机采样技术，提升端侧推理效率。
Qwen	多版本（含QWen2.5-0.5B、QWen3-0.6B等）	阿里巴巴自研，成为淘宝搜索、推荐、AIGC等场景的关键技术基础。
Phi-3	3.8B	微软开发，提供完整的推理工具链，强调在有限资源下的高性能表现。
MobileLLM	50M	采用嵌入共享与分组查询注意力机制，专为移动设备优化。
TinyL-LaMA	1.1B	使用FlashAttention优化内存占用，适合在资源受限环境下运行。
SmolLM	未明确	社区驱动，注重轻量化和高效的推理性能。

因微调后需要经过 AliNNPrivate 转换量化，并在 MNN 引擎中推理，Qwen 系列适配成熟度最高，并且集团内已有相关走通的案例，本着选新不选旧的原则最终在 QWen2.5-0.5B和QWen3-0.6B之间选择了后者作为基础模型。

4.4 Prompt 优化 & 消融实验

我们提供给模型的 Prompt 大致分成 System Prompt + Content Prompt，目前版本使用的 System Prompt：

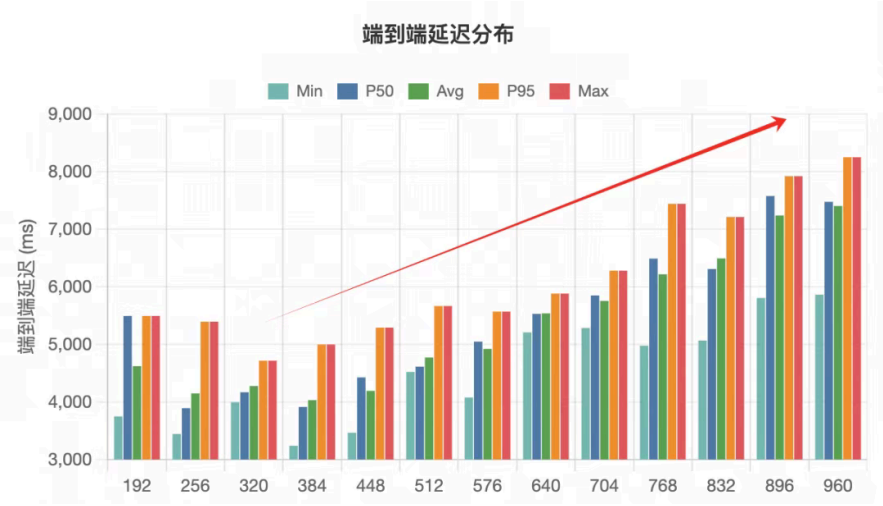
```
"根据用户行为序列判断购买意图并提取核心关键信息
意图分类：
闲逛：无主动购物信号，或跨多个无关品类随意浏览无场景主线
```

```
比价犹豫:同品类内商品对比
场景购物:同一大类下跨子品类围绕统一场景选购(如厨房/露营)
类目说明:
序列中 cat:大类.子类.孙类 为商品类目层级,首段为顶级大类。
判定参考:多个无关大类→闲逛;同一大类跨子类→场景购物;同一完整类目重复→比价犹豫
输出格式:
- 闲逛: 仅输出\"闲逛\"
- 比价犹豫: 输出\"比价犹豫|关键词1;关键词2;...\"
- 场景购物: 输出\"场景购物|关键词1;关键词2;...\"
关键词规则:
- 3-6个,首位为主品类(比价犹豫)或主场景(场景购物)
- 简短名词或形容词,2-6字\n- 按重要性降序排列"
```

Content Prompt 包含用户行为序列，包含搜索、详情、购物车等页面信息。

在端侧核心考虑的点： 保证准确率的情况下尽可能减少 Prompt 长度。

System Prompt 对已有规则细则描述越丰富，模型判断出的意图准确率越高，但System Prompt 长度的增加在端侧会导致推理时间呈线性增加，例如 Vivo S20 上端模型在不同 Prompt 长度下端到端延迟(横坐标是 Prompt 长度)：



以淘特目前使用场景为例，Systemp Prompt 包含“意图分类”、“类目说明”、“判定参考”、“输出格式”、“关键词规则”，这些信息对最终的准确率是否有影响？每个信息影响的范围有多少？这里引入消融实验以同一个数据集做验证，查看将每个部分删除/简化后对准确率的影响。例如我们早期做的一组消融实验，从结果可以看到用户 Profile、类目、商品价格、简版 Prompt、规则对最终准确率的影响范围：

组	总准确率	Δ baseline	闲逛	比价犹豫	场景购物
baseline	84.5%	±0.0pp	66.9%	90.5%	85.8%
no_profile	83.4%	-1.1pp	69.2% (+2.3)	92.1% (+1.6)	80.1% (-5.7)
no_category	82.5%	-1.9pp	66.9% (±0)	90.5% (±0)	80.8% (-5.0)
no_price	83.5%	-1.0pp	60.8% (-6.2)	92.1% (+1.6)	84.4% (-1.4)
minimalprompt	82.4%	-2.1pp	70.0% (+3.1)	89.8% (-0.6)	79.8% (-6.0)
no_rules	83.2%	-1.2pp	68.5% (+1.5)	91.4% (+0.9)	80.8% (-5.0)

以这组实验数据为考量，最终放弃了简化版 Prompt方案，正常版本中也删除了个人 profile。

4.5 模型 LoRA 微调

后训练介绍：

```
大模型微调方法
├── 第一类：全量微调（Full Fine-Tuning）
├── 第二类：参数高效微调（PEFT）
│   ├── 加法类：Adapter、Prefix Tuning、Prompt Tuning
│   ├── 低秩类：LoRA、QLoRA、AdaLoRA、DoRA
│   ├── 选择类：BitFit、LISA
│   └── 乘法类：IA³
└── 第三类：对齐微调（Alignment Fine-Tuning）
    ├── RLHF（PPO）
    ├── DPO
    ├── KTO
    ├── ORPO
    └── GRPO
```

表中是目前后训练方法的整理，方法不逐一介绍，大家有兴趣可以自己了解。其中，全量微调我们资源不允许也没必要，LoRA 是目前 PEFT 中的主流，也是我们目前选择的方案。

LoRA 简单介绍：GPT3 出来之后，微软研究院开始探索怎么在Windows上部署这些模型，包括让用户在 Windows上微调，Edward Hu 等人在 2021 年提出了 LoRA（Low-Rank Adaptation of Large Language Models），将"低秩"这一数学概念巧妙地应用到了模型微调中，开辟了一条全新的技术路线，更多信息大家有兴趣可以查看：https://github.com/microsoft/LoRA。

LoRA 环境搭建：基于 ModelScope + LLaMa-Factory 账号创建、环境搭建、训练参数/数据集配置、量化、上传 OSS等内容可以利用 AI 工具完成，这里不再赘述。

淘特该场景目前微调参数配置分享：

```yaml
# ========== 模型与数据 ==========
model_name_or_path: /mnt/data/.cache/modelscope/models/Qwen/Qwen3-0.6B
trust_remote_code: true
template: qwen3
stage: sft
finetuning_type: lora
dataset: qwen3_06b_0811_intent_key_train-v6
dataset_dir: /mnt/workspace/LLaMA-Factory/data
cutoff_len: 2048
max_samples: 100000
preprocessing_num_workers: 16
packing: false
# ========== LoRA ==========
lora_target: all
lora_rank: 8                    # 维持原 rank（如想配套调大改为 16，alpha 改
lora_alpha: 16
lora_dropout: 0
# ========== 训练超参 ==========
do_train: true
num_train_epochs: 5             # ⭐ 由 3 改为 5
learning_rate: 5.0e-05
lr_scheduler_type: cosine
warmup_steps: 0
max_grad_norm: 1.0
optim: adamw_torch
bf16: true
flash_attn: auto
enable_thinking: false
per_device_train_batch_size: 2
gradient_accumulation_steps: 8
ddp_timeout: 180000000
# ========== ★ 方案 B 核心：验证集 + 自动选优 ==========
val_size: 0.05                  # 从训练集切 5% 作 eval 集
eval_strategy: steps
eval_steps: 100                 # 每 100 步评一次
per_device_eval_batch_size: 4
load_best_model_at_end: true    # 训练结束自动加载 eval_loss 最低的那个
metric_for_best_model: eval_loss
greater_is_better: false        # eval_loss 越小越好
save_strategy: steps
```

```
save_steps: 100                 # 必须与 eval_steps 一致或为其整数倍
save_total_limit: 5             # 仅保留最近 5 个 + best，控制磁盘
save_on_each_node: false
# ========== 日志与输出 ==========
logging_steps: 5
include_num_input_tokens_seen: true
report_to: none
plot_loss: true
output_dir: /mnt/workspace/LLaMA-Factory/saves/Qwen3-0.6B-Thinking/lor
```

完成环境搭建、train.yaml、dateset 配置完成后，可以用 webUI 启动本地 Web GUI 界面调试，也可以直接使用命令行(具体命令可参考前面的 ModelScope 微调实践文档)。

过程中的部分要点：

1、GPU 资源相对珍贵，环境搭建等不依赖 GPU 操作可以先切换到 CPU 环境执行(团队资金充裕的除外)；

2、val / test 数据集完成评测后，可以将结果直接给 AI 生成分析报告，即可以对比各个版本数据趋势，又可以让 AI 分析本次评测准确率高/低原因，并进行下钻；

3、ModelScope 等环境断线重连后部分环境参数需要重新设置，可以写一个 init_env.bash 完成所有启动需要操作，启动环境后执行，提升效率；

4.6 模型剪枝

- 为什么做剪枝

核心诉求是为了减少模型大小和增加推理速度。

- 剪枝前提约束:

剪枝方案的选择的几个约束：

- 改动小 – 产物必须能衔接现有 LLaMA–Factory 和 AliNNPrivate 链路，不改量化和部署逻辑，实现和维护成本要低；
- 指标不动 – 体积、延迟、内存同时降，同时准确率要求保持同样水位；

- 主流方法对照

	特点	评价结果
宽度剪枝（删通道/注意力头/神经元）	要改 `hidden_size`、`num_attention_heads`、`intermediate_size`，打乱权重张量形状。Qwen3 用 GQA，删头还要处理 KV 头分组对齐，量化算子的形状假设可能被破坏，链路改动较大。	❌
稀疏剪枝	把权重矩阵里的小值抹成 0，但端侧 MNN 加手机 CPU 通常吃不到稀疏加速，体积可能降而延迟不一定降，违反三指标同步下降	❌
剪枝加知识蒸馏	产物兼容、精度通常最好，但要搭 teacher-student 管线、调温度和损失权重，成本过高	❌
深度剪枝（整层删除）	产物仍是标准 HF 稠密模型，只是层数变少，`safetensors2mnn` 按层遍历天然兼容。层数线性减少则前向计算量线性减少，CPU 延迟直接下降。	✅

- 淘特剪枝方案

整体采用 ShortGPT 的方案。选层用 Block Influence，定义是 `BI = 1 - E[cos(h_in, h_out)]`，出自 ShortGPT（Men et al., 2024，arXiv:2403.03853）论文的公式 (1)。公式解释：

```
h_in、h_out 是一层的输入输出隐藏态，余弦相似度越接近 1 越接近透传，1 - E[cos] 即平
```

QWen3-0.6B 一共有 28 层 Transformer Block，基于以上公式计算后 BI 的分布整体符合两头高，中间低的规律(Python 版本剪枝代码 100% 由 AI 实现)，淘特微调后 28 层模型 BI 值分布如下：

```
0:0.9103703498840332
1:0.22136521339416504
2:0.13332736492156982
3:0.1306910514831543
4:0.13731390237808228
5:0.11368060111999512
6:0.1134185791015625
7:0.0920405387878418
8:0.10232663154602051
9:0.10968828201293945
```

```
10:0.09634917974472046
11:0.10251200199127197
12:0.07831752300262451
13:0.07851868867874146
14:0.08390378952026367
15:0.09240895509719849
16:0.10546720027923584
17:0.10390979051589966
18:0.09118866920471191
19:0.1061851978302002
20:0.09440338611602783
21:0.1049380898475647
22:0.09798532724380493
23:0.08883386850357056
24:0.06927740573883057
25:0.080806136131286662
26:0.08935767412185669
27:0.5631741285324097
```

根据 BI 值，尝试剪除 6 层、8 层、10 层，结果如下：

	模型层数	云端准确率	Δ	场景购物 Recall	判断
原始	28	93.09%	—	76.64%	—
删 6 层	22	91.94%	−1.15pp	78.50%	通过
删 8 层	20	92.32%	−0.77pp	76.64%（与原始持平）	通过（云端最优）
删 10 层	18	88.87%	−4.22pp	62.62%	跌破阈值

云端裁决是 K=8 最优：掉点最少，比 K=6 多砍两层，场景购物 Recall 与原始持平，端侧速度更快，但真机（量化后）评测数据不及预期：删 8 层在 iOS 上 73.7%，删 6 层 79.8%，差 6.1pp；Android 上删 8 层 76.8%，删 6 层 79.8%，差 3.0pp，速度收益不足以抵消精度损失。生产采用删 6 层，删 8 层不采用。

删层带来结构扰动，之后必须做恢复微调（healing），对比第一次微调修改两处：

- `model_name_or_path` 指向剪枝后的模型而不是原始 base，merge 时的 base 同样换成剪枝后模型；
- `num_train_epochs` 从 5 降到 3（收敛更快，训 5 轮会过拟合到剪枝残留噪声）；其他参数如数据、template、cutoff、LoRA 超参、学习率、验证集选优全部一致。

- 备份方案：Logit 蒸馏

除了剪枝后的 Healing 微调，尝试过的备选方案是 Minitron 式的 Logit 蒸馏，使用过该方案但结果不及 Healing 微调，遂暂时放弃该方案。

Logit 蒸馏大致原理：原始 28 层模型当老师，剪枝后的学生模仿它的概率分布，损失是 `loss = (1−α)·CE + α·KL·T²`，默认参数 α=0.9、T=1.0。通俗点解释：以前教师模型告诉学生模型的是答案，这里告诉的是结果的概率分布。

官方 LLaMA−Factory 不原生支持 logit 蒸馏，用 AI 对 LLaMa−Factory 源码做了修改后实现。这里如果没有 AI Coding，估计直接就放弃了。

4.7 模型层 - 量化：HQQ 与 AWQ 的对比

完成上面所有操作后我们得到的模型可以在 DSW 环境运行或者云端直接部署，但还无法在端侧部署及运行。淘系 App 部署端侧模型目前需要经过 AliNNPrivate 量化和转化(量化是可选项)，之后通过端侧 MNN 推理引擎 SDK 初始化并进行推理。

Qwen3−0.6B 在 FP16 纯权重下大小约 1.2 GB：

```
root@dsw-2144023-7d5d5c7797-gj2sl:/mnt/workspace/models/QWen3_0.6B_finetuned_2026-08-12-15-05-
21# ls -l
总计 1179736
-rw-r--r-- 1 root root        707  8月 12 19:26 added_tokens.json
-rw-r--r-- 1 root root       4168  8月 12 19:26 chat_template.jinja
-rw-r--r-- 1 root root       1366  8月 12 19:25 config.json
-rw-r--r-- 1 root root        214  8月 12 19:25 generation_config.json
-rw-r--r-- 1 root root    1671853  8月 12 19:26 merges.txt
-rw-r--r-- 1 root root        381  8月 12 19:26 Modelfile
-rw-r--r-- 1 root root 1192135096  8月 12 19:26 model.safetensors
-rw-r--r-- 1 root root        613  8月 12 19:26 special_tokens_map.json
-rw-r--r-- 1 root root       5430  8月 12 19:26 tokenizer_config.json
-rw-r--r-- 1 root root   11422654  8月 12 19:26 tokenizer.json
-rw-r--r-- 1 root root    2776833  8月 12 19:26 vocab.json
```

我们用HQQ + INT4 量化后 ~300 MB，体积差四倍：

```
root@dsw-2144023-7d5d5c7797-gj2sl:/mnt/workspace/AliNNPrivate/transformers/llm/export/model202
607302019# ls -l
总计 310872
-rw-r--r-- 1 root root      1395  7月 30 20:19 config.json
-rw-r--r-- 1 root root      1072  7月 13 19:16 export_args.json
-rw-r--r-- 1 root root      4722  7月 13 19:16 llm_config.json
-rw-r--r-- 1 root root    397208  7月 13 19:16 llm.mnn
-rw-r--r-- 1 root root    793459  7月 13 19:16 llm.mnn.json
-rw-r--r-- 1 root root 313924950  7月 13 19:16 llm.mnn.weight
-rw-r--r-- 1 root root       555  7月 30 20:19 MD5_Verify.json
-rw-r--r-- 1 root root   3193569  7月 30 20:19 tokenizer.txt
```

量化命令（基于 hqq）：

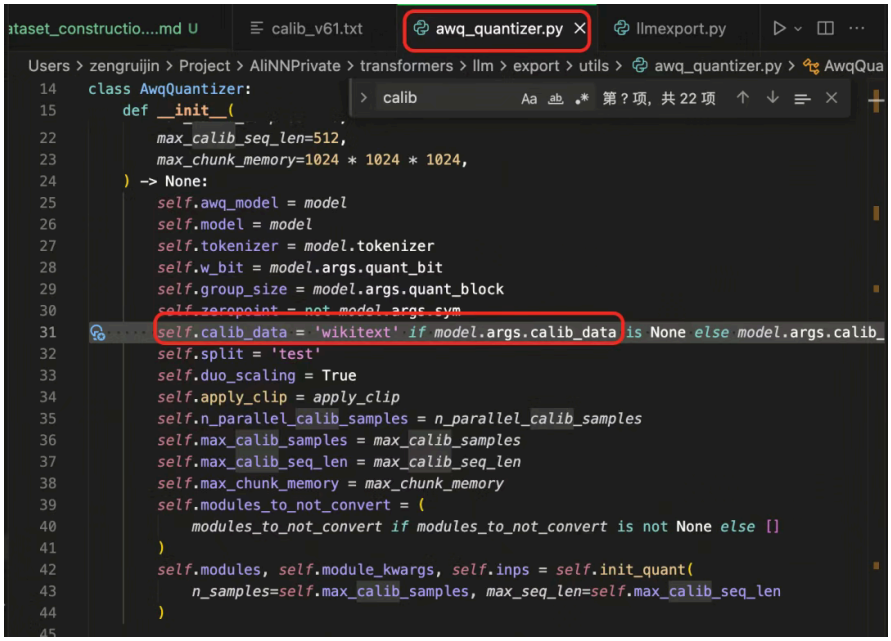
```
python llmexport.py \
    --path /mnt/workspace/models/QWen3_0.6B_finetuned_2026-08-12-15-05-
    --export mnn \
```

```
    --hqq
```

- HQQ 和 AWQ

	量化特点	量化导出时间	产物大小	真机准确率差异(同一数据集评测对比)
HQQ	• HQQ 对模型中所有权重做一样规则的量化，对所有权重矩阵分组，用半二次优化把每组的 scale 和 zero point 求得更准；	分钟级	280M	79.8%
AWQ + quant_block = 32	• AWQ 理论中认为约有 1% 的权重其对 output 的影响要比其他权重重要，将这些少数权重找到并对其加权使模型因量化带来的损失减少； • 建立自己的词典，先喂一批真实数据观察哪些通道是要害再定向保护； • OmniQuant 用梯度训练量化参数，精度上限最高但导出慢；	小时级	380M	91.8%

- AliNNPrivate 自带的 AWQ 量化方法用的字典是一段英文 wiki 字典，需要根据自己数据集创建字典，并修改代码设置成自己字典路径:



- AWQ 导出速度变慢(从小于 1 分钟 -> ~40分钟)、产物大小增加较多(288M -> 376M)，但真机 benchmark 准确率提升明显（79.8% -> 91.8%）；

导出后压缩对应文件，可以手动下载并上传 OSS 或根据 OSS 提供的接口在 ModelScope 提供的 DSW 环境中通过脚本实现上传。

4.8 推理 & 应用

端侧意图推理技术架构：



- 推理链路架构和核心流程概述

端侧意图推理链路采用“配置驱动、可观测、可灰度”的分层架构。

- 配置中心 – 配置中包括：模型地址、总开关、用户画像、疲劳度以及触发规则；
- 流程管线 – 负责下载管理和推理前检测；
- 推理链路 – 负责调用 MNN SDK 进行意图推理及意图分发；
- 监控系统 – 监控系统把每个关键节点的状态与性能指标上报到远程看板，同时在 Debug 包内保留一份本地环形缓冲日志；

一次完整的端侧意图推理核心流程包括：

1、**配置拉取**：应用启动或账户切换后，配置中心先读本地缓存让链路尽快具备可用状态，再异步请求服务端刷新，只有配置 MD5 变化才会更新本地数据；

2、**下载管理**：随后模型资源管理器根据配置中的模型地址判断是否需要下载，满足存储空间(Orange 配置，>=1 G 剩余空间)等条件才会在后台启动下载与解压；

3、**用户行为记录**：App 启动后用户行为事件经过白名单过滤后进入规则引擎，引擎会按照序列、统计、组合等条件持续匹配，当行为累计满足某条规则时(例如序列中>=3个详情 PV)，便根据用户路径生成一段标准 prompt 并触发推理请求；

4、**推理准入**：请求先经过五道闸，总开关是否打开、当前设备是否在允许范围、是否超过疲劳度限制、模型是否就绪、以及剩余内存/发热等运行时状态是否正常；只要任一条件不满足，本次推理立即终止并记录跳过原因。

5、**意图推理 & 分发**：全部通过5道闸判断后调用端侧推理接口执行推理，推理结束后剥离模型输出中的思考片段，通过关键词命中得到“比价犹豫”“场景购物”“闲逛”或“未知”四种意

图；

6、意图分发：其中只有前两类会分别分发为 shopping_hesitate_trigger(比价犹豫）和 shopping_scene_trigger(场景购物）事件，交由业务方监听消费；

7、**数据上报**：整条链路的配置、下载、触发、推理、意图解析和分发结果都会作为监控数据上报，用于构建漏斗看板和后续优化。



完成端模型量化及上传 CDN ，客户端下载成功后，随着用户使用便进入了推理环节。

端侧推理耗时可以简化成 Prefill 阶段 & Decode 阶段，以 QWen3-0.6B 模型为例子简单介绍两个核心环节：

- Prefill(预填充)：

- 当你输入一段 prompt（比如 500 个 token）时，模型需要"阅读"这整段文字。Prefill 阶段会将所有 token **一次性** 送入模型，计算出每个 token 在 28 层 Transformer 中的表示。

```
输入：[token_1, token_2, ..., token_500]
          ↓ 一次性送入
     ┌──────────────────┐
     │  Transformer     │  × 28 层
     │  (并行处理)       │
     └──────────────────┘
          ↓
输出：第一个生成 token
```

- **核心计算特征**：大矩阵乘法。每层包含 7 个矩阵乘（Linear：q/k/v/o_proj + gate/up/down_proj），例子中输入矩阵形状为 `[500, 1024]`（500 个 token，每个 1024 维），与权重矩阵相乘。这是一个**计算密集型操作**。

- Decode：

- Prefill 完成后，模型开始逐个生成输出 token。每生成一个 token，都要运行一遍完整的 28 层 Transformer，但此时输入只有 **1 个 token**；

```
                        ┌──────────────┐
已有的 KV Cache ──→     │ Transformer  │ ──→ 生成 token_1
+ 当前 token_0          │ (28 层)      │
                        └──────────────┘
                              ↓ 把 token_1 作为新输入
                        ┌──────────────┐
已有的 KV Cache ──→     │ Transformer  │ ──→ 生成 token_2
+ 当前 token_1          │ (28 层)      │
                        └──────────────┘
                              ↓ 如此反复...
```

- 核心计算特征：矩阵-向量乘法。输入变成 `[1, 1024]`，权重不变。计算量骤降，但每次都要从显存加载全部权重。这是一个**访存密集型**操作。

5.1 指标体系定义

要做优化先建基线。Prefill 与 Decode 的瓶颈在不同阶段，Prefill 是大矩阵乘，算力受限；Decode 是矩阵向量乘，每生成一个 token 都要把全部权重完整读一遍，带宽受限。指标因此拆成四类共十五项，按 P0/P1/P2 排优先级排列，包括 TTFT、Decode 吞吐、峰值 RSS、模型加载耗时为 P0，分别对应等待时长、出字连续性、被杀风险与首次打开耗时。具体指标如下：

分类	指标	采集方式	粒度级别	连续推理影响
A 速度/吞吐	Prefill tok/s	实时可测	单次推理级	热降频/内存压力下随轮次衰减
	Decode tok/s	实时可测	单次推理级	热降频/内存压力下随轮次衰减
	TTFT（首 Token 延迟）	实时可测	单次推理级	KV-cache 增长可能导致延迟上升
	ITL（Token 间延迟）	实时可测	单次推理级	热降频下逐 token 延迟上升
	E2EL	实时可测	单次推理级	受 Prefill + Decode 共同衰减影响
	模型加载时间	实时可测	单次推理级	—
B 内存/资源	峰值内存	实时可测	单次推理级	多轮推理累积是否持续增长
	模型加载内存	实时可测	单次推理级	—
	推理时内存增量	实时可测	单次推理级	多轮推理是否累积不释放
	连续推理内存泄漏率	后处理分析	系统级	本身即连续推理派生指标
	CPU 使用率	实时可测	单次推理级	热降频后 CPU 调度变化
C 环境/稳定性	热状态	实时可测	系统级	本身即连续推理核心观测项
	持续推理吞吐衰减率	后处理分析	系统级	本身即连续推理派生指标
D Prompt 长度影响	Prefill 速度 vs Prompt 长度	后处理分析	批次级	—
	内存占用 vs Prompt 长度	后处理分析	批次级	—

5.2 真机端模型 Benchmark 建设

Benchmark 核心分成 2 个阶段：**Prompt 长度分档测试 & 连续推理压力测试**

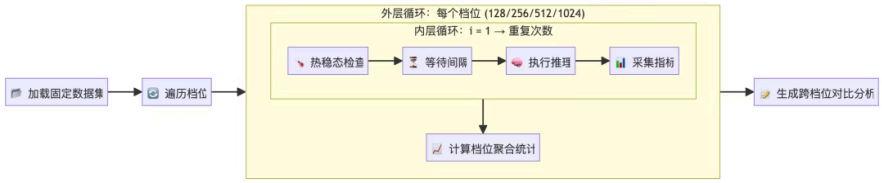
- 长度分档测试

观察不同 prompt 长度对推理性能（prefill/decode 吞吐量、TTFT、端到端延迟）、内存占用（peak RSS、phys footprint、内存增量）和设备温控的影响。基于 val 数据集根据长度分别挑选 128 / 256 / 512 / 1024 区间附近的数据集 item 组成分档测试用例。

测试参数：

参数	说明	默认值	可自定义
档位	prompt token 长度分档	128 / 256 / 512 / 1024	✓
重复次数	每档重复执行次数	10	✓
KV Cache	每次重置（独立请求模式）	重置	-
热稳态门控	严重/临界热状态时暂停	启用	✓
轮次间隔	两次推理间的冷却间隔	5 秒	✓
最大输出 tokens	每次推理生成上限	128	✓

测试流程：



每次测试结果汇总到 report 文件夹中，连续多次评测可以横向对比；

- 连续压力测试

评估模型在长时间连续推理负载下的：

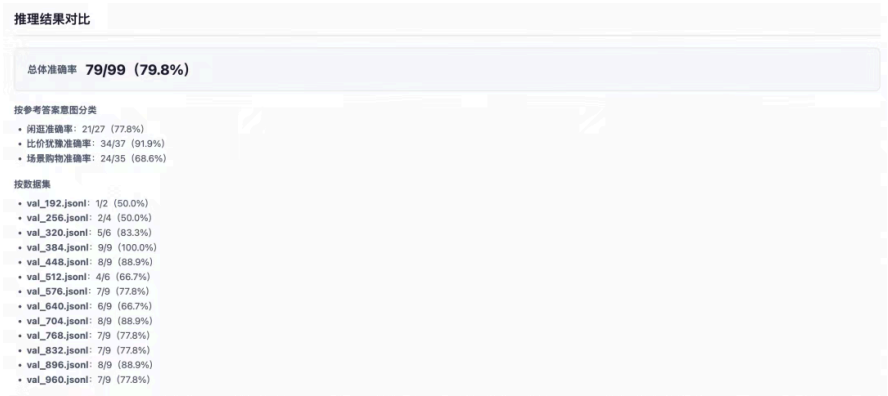
- 性能稳定性（吞吐量是否退化）
- 内存健康（是否存在泄漏）
- 热行为（设备是否过热降频）

测试参数：

参数	说明	默认值	可自定义
连续次数	背靠背执行的总轮数	50	✓
KV Cache 模式	保留跨轮 / 每轮重置	可选	✓
热稳态门控	关闭（模拟真实持续负载）	关闭	✓
轮次间隔	0 秒（背靠背）	0	✓
使用样本	固定中位长度 prompt	1 条	-

Benchmark 报告样例：





5.3 优化手段 & 优化效果

优化方法	具体优化内容	优化效果
简化 Prompt	将 Prompt 长度缩短，减少 Prefill 时间。	高端机线上均值减少 400ms；
模型剪枝	将模型从 28 层修剪成 22 层；	高端机线上均值减少 800ms；
Prefix cache	因为淘特场景 systemp prompt 固定，因此可以缓存这部分数据，减少 Prefill 阶段时间。	• iOS Prefix cache 上线后整体优化 100ms； • Android ◦ 整体推理时间： 4.59 -> 3.47(-1.12s) ◦ 高端机： 2.86 -> 2.32(-0.54s) ◦ 中端机： 5.9 -> 4.59(-1.31s)
增量推理(iOS iPhone 17 系列设备灰度中)	用户每产生一个行为便完成该序列新 Token 的 Prefill 相关并缓存数据，待最后触发序列到达时一起推理。相当于将一个长序列的推理分解成多次 Prefill + 一次Decode；	iOS iPhone 17：优化 1.2s（1.9s -> 0.6s）

06 业务落地的场景建设

淘特目前落地了基于用户意图对首猜未曝光卡片做重排序，以及判断用户有购物意图但未购物快流失时给用户发权益挽留。

这里针对端模型 + 权益的场景详细展开，架构和流程如下：



落地场景中端侧模型识别、权益模块触发、服务端统一管控，各模块分工概述：

- **端侧模型**在用户浏览商品过程中基于本地行为序列识别意图，一旦输出有效意图，立即发送携带 sceneCode、modelId、modelVersion 和 eventId 的标准广播 ACTION_CLIENT_MODEL_INTENT_HIT；
- **端侧权益模块**不参与权益决策、实验分组、命中计数或延迟处理。权益模块监听该广播后，读取 llm_benefit_dispatch_runtime 配置完成 sceneCode → triggerGroup 映射，按会话累计同组命中次数；当达到第 N 次时启动延迟 X 秒，延迟结束时调用端侧 hasOrderedSince 接口确认用户是否已下单；若未下单则调用 BXPEIEngine.triggerPeiDecision 进入 PEI directTrigger 链路；
- **服务端**负责从全人群中切出 40% 端模型验证人群，并在此人群中按实现组/对照组稳定分桶，端侧不感知，最终是否发放权益由后端根据上传的用户行为记录交由云端大模型判断并决定。如判断需要发券，则通过 UBA 动线系统给用户弹出权益 Pop 通知用户有新权益到达。

目前该场景还在实验和持续放量中，从阶段性数据可以看到对比对照组数据，实验组通过端模型意图识别并发券增量 GMV 持续正向(同样投资1元实验组增加的 GMV)，近一周数据：

端模型汇总数据

日期	试验桶DAU	AB增量ROI_不含token	AB增量ROI_含token	AB增量GMV	GMV提升比例	访购率提升pt
20260831		3.3068	3.2203		7.94%	1.92%
20260830		4.0496	3.9442		10.53%	2.21%
20260829		3.6799	3.5789		8.89%	1.64%
20260828		5.3015	5.1642		14.03%	1.57%
20260827		2.3481	2.2812		5.43%	1.45%
20260826		3.6757	3.5691		7.99%	1.42%
20260825		3.8302	3.7196		8.66%	1.44%

- 隐藏成本：CDN

端模型对比云端模型没有存储、部署/调用、Token 等相关成本，但端侧 App 每次下载模型会产生 CDN 费用，从淘特数据看(单模型 Zip 包280M)，其实是一笔不小的成本。为优化下载效率，部分实现如下：

优化方案	具体实现
按需下载	原下载任务在 idle 阶段，机型适配后即下载，修改成用户有触发推理后在首次调用推理链路时进入下载。
机型屏蔽	部分设备本身因硬件及系统等原因推理失败率接近 90%，先灰度，过程中监控发现后直接屏蔽机型，杜绝该机型其他设备下载； 屏蔽既考虑失败率也考虑推理量级；
剪枝	28 层模型接近 315M，剪枝成 22 层后大小到达 288M，大小减少约 10%；

07 后续规划和思考

到目前为止，我们验证了端模型在 App 侧业务中应用的可行性，以及对部分业务指标的正向作用。后续端侧模型规划包括：

- **模型能力拓展** – 目前的意图分类仅针对进端购物的群体，端侧有大量互动型的用户只玩/逛但是不买，后续将结合端侧模型能力更好理解这部分用户，增加购买率的转化；
- **模型准确率提升** – 探索并走通 剪枝 + AWQ + quant_block=32 链路，实现真机准确率从目前80%水位提升到 90%；
- **推理性能优化** – 增量推理能力扩大放量并最终全量部署，将推理速度压缩到 1s 以内；
- **端侧 Agent 建设** – 在现有单次会话推理的基础上增加记忆，从基于用户行为序列推理升级为基于用户购物习惯的推理，更精准贴近用户意图；

08 团队介绍

本文作者仄平，来自淘天集团–互动&体验团队，该团队工作覆盖淘特 App 导购、交易、搜索、用增及 AI Native。团队持续探索 AI 原生体验，尝试把玩法创意、内容生产、研发交付和运营迭代连接起来。

¤ 拓展阅读 ¤



大淘宝技术

大淘宝技术官方账号

966篇原创内容

公众号

阅读 3645

留言

写留言

还没有留言，来写首评